Quad-Dimensional DataMatrix

Request for Discussion: Hexadecimal GIF States and App-Held Sequences

João Paulo De Luca Jr Jorge Luiz Espanhol

4 November 2026 · RfD 0.2.0

Abstract

Quad-Dimensional DataMatrix is proposed as a discussion framework for hexadecimal data represented by indexed GIF states. The working profile uses a 256 by 256 pixel canvas, 64 by 64 effective modules, up to 64 hexadecimal palette entries, and up to 64 frames. Four logical coordinates identify horizontal position, vertical position, temporal state, and dimension. Their maximum Cartesian catalogue contains 16,777,216 candidate tuples. A dense sequence selects one of 64 alternatives at each module and therefore carries up to 192 KiB of gross payload in a 64-frame cycle, nominally eight seconds at 8 fps. An app-held sequence supports a second interaction: advance at a chosen cadence, pause on user selection, and copy the state's hexadecimal code for a receiving service. The paper develops the counting model, separates the two timing roles, and examines one-to-one Unicode-to-hexadecimal mappings as candidates for a future dictionary or key. A 16-frame case study and a local bilingual viewer accompany the discussion. Scope and Limitations consolidates the qualifications. The document invites refinement before a specification is proposed.

1 Purpose and discussion status

Quad-Dimensional DataMatrix is a Request for Discussion (RfD) by **João Paulo De Luca Jr and Jorge Luiz Espanhol**. It brings together a four-coordinate catalogue, indexed GIF transport, hexadecimal values, and app-held state selection.

The immediate objective is to make the proposed objects countable and testable. A reader should be able to identify what a state contains, how the app selects it, what the user copies, and which interpretation of a hexadecimal value is being proposed. A later stage may develop a specification from the discussion.

The working vocabulary is:

Term	Meaning in this RfD
Stored pixel	One index in the stored image
Effective module	A repeated-index 4×4 block
Dimension	An active hexadecimal palette identifier
State	A complete matrix at one temporal index
Sequence	An ordered set of up to 64 states
Catalogue tuple	A candidate (x, y, t, d) tuple
Code	A state-associated value for consumption
Key	A dictionary entry or secret proposed for discussion

English is the default language of the repository and manuscript. A complete Portuguese edition is maintained alongside it. The qualifications are collected in Scope and Limitations; the argument below develops the working model directly.

2 Working GIF profile and hexadecimal values

2.1 Geometry and active entries

The current profile is described by

$$W_p = H_p = 256, \qquad s = 4, \qquad W = H = 64, \qquad K \leq 64, \qquad F \leq 64.$$

The stored frame has $W_p H_p = 65{,}536$ pixels. Its independent module count is

$$WH = \frac{W_p H_p}{s^2} = 4{,}096.$$

The factor $s^2 = 16$ repeats each selected index across a square module. It determines the rendering geometry, while WH determines the number of independent module choices.

Each active palette entry is a six-digit hexadecimal value such as `#12ABEF`. For state t, the mapping is

$$P_t : \{0, \ldots, K-1\} \longrightarrow \{\texttt{\#000000}, \ldots, \texttt{\#FFFFFF}\}.$$

The public relationship is

$$\text{dimension} \longleftrightarrow \text{palette index} \longleftrightarrow \text{hexadecimal value}.$$

The current adapter treats the ordered palette as the codebook. Palette order can change between states while each entry remains a hexadecimal value. GIF89a supplies the indexed table and image-block container.

2.2 Representation and table cost

A six-digit hexadecimal value has a 24-bit integer domain:

$$16^6 = 2^{24} = 16{,}777{,}216.$$

The active alphabet has K entries, so choosing an entry costs $\log_2 K$ bits in the unrestricted counting model. At $K = 64$, that choice carries six bits.

A GIF table with K entries occupies

$$B_{\text{table}} = 3K \text{ bytes}.$$

At 64 entries, the table occupies 192 bytes. If every state supplies its own table, the table allocation is $3KF$ bytes. Textual hexadecimal descriptions and the encoded container have their own storage budgets. The model keeps these quantities separately observable.

3 The four-coordinate catalogue

3.1 Cartesian catalogue and selected state

Let

$$X = \{0, \ldots, W-1\}, \quad Y = \{0, \ldots, H-1\}, \quad T = \{0, \ldots, F-1\}, \quad \Delta = \{0, \ldots, K-1\}.$$

The proposed four-coordinate catalogue is

$$\Omega = X \times Y \times T \times \Delta, \qquad |\Omega| = WHFK.$$

For the maximum profile,

$$|\Omega| = 64^4 = 16{,}777{,}216.$$

A dense sequence chooses a dimension at every position and temporal state:

$$D : T \times Y \times X \longrightarrow \Delta.$$

Equivalently, define a four-index indicator tensor

$$A[t, y, x, d] = \begin{cases} 1, & D[t, y, x] = d, \\ 0, & \text{otherwise.} \end{cases}$$

The selection condition is

$$\sum_{d=0}^{K-1} A[t, y, x, d] = 1.$$

The catalogue enumerates every candidate tuple; the dense sequence selects one dimension for each of its WHF positions. This gives a precise meaning to the project's four logical coordinates.

3.2 A hexadecimal catalogue address

With all four coordinate ranges bounded by 64, a row-major catalogue address can be written as

$$a = (((64t + y)64 + x)64 + d).$$

It spans

$$0 \le a \le 64^4 - 1 = \texttt{0xFFFFFF}.$$

The inverse is

$$d = a \bmod 64, \qquad x = \lfloor a/64 \rfloor \bmod 64,$$

$$y = \lfloor a/64^2 \rfloor \bmod 64, \qquad t = \lfloor a/64^3 \rfloor.$$

An address uses 24 bits, with six bits for each coordinate. The implementation exposes the address as a six-digit hexadecimal integer. This address is useful for discussing dictionaries, sparse events, and state-local references.

4 Capacity and nominal rates

4.1 From choices to bits

For $N = WHF$ independently selectable modules, the number of dense sequences is

$$|\mathcal{D}| = K^N.$$

A binary message of length L requires 2^L distinguishable encodings. Therefore

$$2^L \le K^N \quad \Longrightarrow \quad L_{\max} = \lfloor WHF \log_2 K \rfloor.$$

At the maximum profile,

$$N = 64^3 = 262{,}144, \qquad \log_2 K = 6,$$

$$L_{\max} = 1{,}572{,}864 \text{ bits}, \qquad B_{\max} = \left\lfloor \frac{L_{\max}}{8} \right\rfloor = 196{,}608 \text{ bytes} = 192 \text{ KiB}.$$

The per-state gross payload is

$$B_{\text{state}} = \frac{64^2 \cdot 6}{8} = 3{,}072 \text{ bytes} = 3 \text{ KiB}.$$

The number of possible complete sequences is exponential in N. The binary payload is its logarithm and grows proportionally to F.

4.2 Catalogue rate and payload rate

Let f be the nominal number of displayed states per second. A full cycle has duration

$$\tau = F/f.$$

For $F = 64$ and $f = 8$, $\tau = 8$ seconds. The catalogue traversal budget is

$$R_\Omega = \frac{WHFK}{\tau} = WHKf = 2{,}097{,}152 \text{ candidate tuples/s}.$$

This is the intended interpretation if the abbreviation **cps** is used for the catalogue in this RfD. Two additional rates describe selected data:

$$R_{\text{modules}} = WHf = 32{,}768 \text{ modules/s},$$

$$R_{\text{bits}} = WHf \log_2 K = 196{,}608 \text{ bits/s} = 24 \text{ KiB/s}.$$

Measurement	Maximum profile
Four-coordinate candidates	16,777,216 per cycle
Candidate-tuple budget	2,097,152 per second at 8 fps
Selected modules	262,144 per cycle
Gross payload	192 KiB per cycle
Gross payload rate	24 KiB/s at 8 fps

For a future framing design with h reserved bits per state and code rate r, a first payload budget is

$$B_{\text{net}} = \left\lfloor \frac{rF(WH \log_2 K - h)}{8} \right\rfloor, \qquad 0 < r \leq 1.$$

This expression assumes fixed per-state reservations followed by an aggregate code of rate r. A different placement of reservations yields a different budget. The eventual layout is a discussion item.

5 Time in the container and time in the app

5.1 Transport cadence

GIF delays are stored in hundredths of a second. A nominal 125 ms state period can be represented by alternating 120 and 130 ms delays. For an even frame count,

$$\sum_{i=0}^{F-1} \delta_i = 125F \text{ ms}.$$

The prototype quantizes cumulative boundaries:

$$\delta_i = 10 \left(\left\lfloor \frac{(i+1)p}{10} \right\rfloor - \left\lfloor \frac{ip}{10} \right\rfloor \right) \text{ ms},$$

where p is the requested period in milliseconds. At $p = 125$ and $F = 64$, the stored delays sum to 8,000 ms.

5.2 App-held sequence and selection

The proposed interaction places the sequence in the app. The user or a receiving service selects or consumes one state. The app can advance at a chosen interval Δ, including one state per minute.

One possible state selector is

$$i(\theta) = \left\lfloor \frac{\theta - \theta_0}{\Delta} \right\rfloor \bmod F.$$

If the user selects at time θ_c, the frozen index is

$$i_c = i(\theta_c).$$

A chosen state-to-code mapping g supplies the clipboard value:

$$c = g(D_{i_c}).$$

The local study viewer uses the state payload's canonical hexadecimal string for g. It defaults to $\Delta = 60$ seconds. A 64-state cycle then spans 64 minutes. The 8 fps transport example and the one-frame-per-minute interaction are two uses of the same ordered states.

Event	App action	Receiving side
Load	Keep the sequence and its mapping locally	Await a code
Advance	Display the next state	Observe or request selection
Click	Stop advancement and freeze the displayed state	User selects the current state
Copy	Copy the associated hexadecimal code	User, service, or site consumes it
Resume	Continue when requested	Another state may be selected

The example renders each selected state as an independent GIF. Clicking clears the app timer before writing to the clipboard. This makes the selected image and copied code refer to the same state.

5.3 Relationship to TOTP

The comparison with TOTP concerns the ownership of evolving state and the role of the receiving verifier. In TOTP, an app holds a secret and computes a code using a temporal counter. In this proposal, the app holds or obtains a sequence and exposes a selected state's code.

The receiving service is the consumer or verifier in both interaction patterns. The RfD invites discussion of how g, provisioning, state renewal, and acceptance rules should be chosen for this sequence-based model. The Clipboard API provides the user-triggered copy operation used by the local example.

6 Unicode and one-to-one hexadecimal mappings

6.1 The scalar domain

The Unicode scalar domain is

$$\mathcal{U} = \{0, \ldots, \texttt{0x10FFFF}\} \setminus \{\texttt{0xD800}, \ldots, \texttt{0xDFFF}\}.$$

Its cardinality is

$$|\mathcal{U}| = \texttt{0x110000} - \texttt{0x800} = 1{,}112{,}064.$$

The proposal's "approximately one million codes" is naturally related to this domain. UTF-8 is a variable-length encoding of those scalar values using one to four octets.

A direct hexadecimal representation is

$$h(u) = \mathrm{hex}_6(u).$$

It is injective: $h(u_1) = h(u_2)$ implies $u_1 = u_2$. The representation fits inside the 24-bit value domain already used by the catalogue and palette descriptions.

6.2 Three candidate interpretations

Direct palette dictionary. A scalar value can be associated with a six-digit hexadecimal entry. A frame may use at most K distinct scalar entries in this interpretation. An adaptive dictionary could change the active entries between states. Repeated uses of an entry remain ordinary repeated module choices.

Four-dimension scalar representation. A fixed alphabet of 64 dimension identifiers can represent any scalar with four base-64 digits:

$$u = 64^3 d_0 + 64^2 d_1 + 64 d_2 + d_3.$$

The digits are

$$d_j = \left\lfloor \frac{u}{64^{3-j}} \right\rfloor \bmod 64, \qquad j = 0, 1, 2, 3.$$

Since

$$64^3 < |\mathcal{U}| < 64^4,$$

four fixed-length base-64 digits are sufficient, while three cannot identify every scalar. The prototype provides this reversible mapping.

Catalogue-address dictionary. The scalar's integer can also be embedded in the 24-bit catalogue address domain. The inverse address equations produce a coordinate tuple. This supports a one-to-one dictionary entry or an addressed event. A sequence of addressed events also needs a representation of order and multiplicity; these become explicit design choices.

Scalar example	Hexadecimal scalar	UTF-8 octets	Four dimension identifiers
Latin capital A	`0x000041`	`41`	`00 00 01 01`
Latin small a with tilde	`0x0000E3`	`C3 A3`	`00 00 03 23`
Euro sign	`0x0020AC`	`E2 82 AC`	`00 02 02 2C`
Grinning face	`0x01F600`	`F0 9F 98 80`	`00 1F 18 00`

6.3 Text budget and the eventual key

The four-dimension representation uses 24 bits per scalar:

$$B_{\mathrm{scalar}} = 3 \text{ bytes}, \qquad C_{\mathrm{state}} = \left\lfloor \frac{4096}{4} \right\rfloor = 1024 \text{ scalars}.$$

A maximum 64-state cycle holds 65,536 such fixed-length scalar representations before framing. Relative to UTF-8, the scalar mapping spends more bits on single-octet values and fewer bits on four-octet values. The useful comparison therefore depends on the text repertoire and the dictionary design.

The RfD leaves the term “key” open to a concrete interpretation: a public dictionary, a provisioned private dictionary, an index into external state, or an input to a cryptographic construction. The one-to-one mapping establishes a representation; its eventual role can be selected through discussion.

7 Reproducible case study and app example

7.1 The 16-state study

The published demonstration deliberately contains 16 states. Each state uses the current 256 × 256 pixel canvas, 64 × 64 effective modules, and 64 hexadecimal palette entries. The ordered palette is rotated between states. The payload is synthetic and reproducible from seed 20261104.

Its gross data budget is

$$B_{16} = \frac{64 \cdot 64 \cdot 16 \cdot 6}{8} = 49{,}152 \text{ bytes} = 48 \text{ KiB}.$$

The decoder recovers the original payload exactly. The report records the original and recovered SHA-256 values, the GIF hash, environment versions, hexadecimal palettes, state count, and nominal duration.

Property	Measured study
Stored image	256 × 256 pixels
Effective state	64 × 64 modules
Frames	16
Palette entries per state	64
Stored nominal cycle	2,000 ms
Recovered payload	49,152 bytes
Encoded GIF size	383,644 bytes
Recovery	Byte-exact

The encoded file size is a measured container size. The gross payload count comes from the number of independent selected dimensions.

7.2 Implementation and checks

The code accepts hexadecimal payload octets and hexadecimal palette entries. It writes complete indexed frames, including repeated states, and reads the stored index data through the declared palette order. The decoder checks that each effective module contains 4 × 4 identical indices.

The test suite exercises:

- Exact recovery for varying payload lengths, including zero octets.
- Preservation of changing palette order.
- Distinct frame records for identical states.
- The 64-frame, 192 KiB working-profile boundary.
- One state per minute timing.
- Bounds for frames, palettes, and hexadecimal input.

- Truncated-container rejection and inconsistent-module rejection.
- Four-coordinate address inversion and Unicode scalar boundaries.

The local viewer defaults to English, offers Portuguese, and pauses before copying the current state. It includes manual copying when the direct clipboard operation is unavailable. The complete copied example code contains 6,144 hexadecimal digits for one 3 KiB state.

The repository provides the case-study generator, results, viewer, both manuscript sources, and build commands.

8 Scope and Limitations

This section consolidates the interpretation and boundaries of the RfD.

Discussion status. The profile, vocabulary, and mappings are working proposals. Framing, code length, interoperability, provisioning, and a verification protocol are subjects for discussion before a specification.

Logical dimensions and counting. The four axes enumerate candidate tuples. One dimension is selected per spatial-temporal position. The 16,777,216 catalogue entries and the 192 KiB gross selected payload measure different objects. “cps”, when used, denotes candidate tuples per second.

Hexadecimal interface and container. Hexadecimal is the public representation of palette values and codes. GIF89a supplies its internal serialization. A change of numeral representation does not create additional entropy. The current adapter uses complete opaque frames and power-of-two palettes with at most 64 entries. The original payload length is supplied by the caller.

Budgets and timing. Header fields, dictionaries, authentication data, redundancy, and container storage require separate budgets. The 8 fps example states nominal playback timing; the app’s state cadence is independent. The study does not measure network transfer speed, generative-model throughput, or physical capture.

Authentication. The app example contains public synthetic states and demonstrates selection and copying. A repeated finite sequence needs a chosen provisioning and renewal design for authentication use. Confidentiality, challenge binding, expiry, replay handling, and rate limiting belong to that design. TOTP and AEAD are references for the discussion.

Unicode and dictionaries. Scalar values, assigned characters, grapheme clusters, UTF-8 octets, active palette entries, and keys have different domains. The scalar mappings preserve scalar identity; normalization, ordering, multiplicity, and dictionary storage require explicit choices in a text or key design.

Evidence. The recorded results apply to the current exact indexed-file path. They support the specified counting and recovery claims and do not establish the security of a later protocol.

9 Questions for discussion

1. Which applications benefit from the four-coordinate catalogue: dense data, addressed events, state selection, or dictionary lookup?
2. Which hexadecimal object should be called a key, and how should it be provisioned?
3. Should text use UTF-8 octets, four-dimension scalar values, or adaptive state dictionaries?
4. How should the selected code be associated with a service, challenge, or acceptance window?
5. Which applications need fast transport, slow app cadence, or explicit user-driven advancement?
6. What should renew a sequence after a cycle or after consumption?

7. Which fields are necessary before a specification can be proposed?

10 Authorship and assistance

João Paulo De Luca Jr and Jorge Luiz Espanhol are the co-authors of this RfD and the associated discussion proposal. ChatGPT/Codex, OpenAI, assisted with mathematical exposition, translation, implementation, execution, and document preparation. The authors retain responsibility for the research direction and decisions.

11 Linked technical references

1. CompuServe, Graphics Interchange Format Version 89a, reproduced by W3C.
2. Pillow, GIF image-file support.
3. Unicode 18.0, Chapter 3: Conformance, especially §3.9.
4. RFC 3629: UTF-8, a transformation format of ISO 10646.
5. RFC 6238: TOTP, Time-Based One-Time Password Algorithm.
6. RFC 5116: An Interface and Algorithms for Authenticated Encryption.
7. W3C, Clipboard API and events.